Final Report

Summer of Science 2025: Artificial Intelligence & Machine Learning Theory

Rahul Kumar (23B1212)

Mentor: Priyanshu Kumar

Week 1: Foundations of AI/ML and Essential Mathematics

1. Introduction to AI, ML, and DL

Artificial Intelligence (AI) is a branch of computer science that aims to create machines capable of intelligent behavior. In recent decades, AI has grown from symbolic reasoning systems to statistical methods and neural networks.

Machine Learning (ML), a subset of AI, refers to algorithms that improve automatically through experience and data. Instead of being explicitly programmed, these algorithms learn from patterns and insights in the data.

Deep Learning (DL) is a further specialization under ML, leveraging deep neural networks (DNNs) with many layers. These models are particularly good at extracting abstract features from high-dimensional inputs like images and text.

Historical Context

- 1950s: Alan Turing introduces the Turing Test.
- 1956: Term "Artificial Intelligence" coined at the Dartmouth Conference.
- 1980s: Rise of expert systems.
- 2000s: Emergence of big data and GPU computing fuels ML/DL growth.
- 2010s–present: Breakthroughs in computer vision, NLP, and autonomous systems.

2. Categories of Machine Learning

Machine learning can be broadly categorized as:

1. **Supervised Learning:** Learns a mapping from inputs to labeled outputs.
2. **Unsupervised Learning:** Finds hidden structures in unlabeled data.
3. **Semi-supervised Learning:** Combines small amounts of labeled data with large amounts of unlabeled data.
4. **Reinforcement Learning:** Agents learn optimal policies through trial and error, receiving rewards or penalties.

Examples:

- Supervised: Email spam detection, house price prediction
- Unsupervised: Customer segmentation, topic modeling
- Reinforcement: AlphaGo, autonomous driving

3. The ML Lifecycle / Pipeline

The machine learning workflow typically consists of:

1. **Problem Definition and Goals:** Understanding the business or research question.
2. **Data Collection:** From sensors, logs, web, surveys, etc.
3. **Data Cleaning:** Handle missing values, outliers, inconsistencies.
4. **Exploratory Data Analysis (EDA):** Visualize distributions, correlations.
5. **Feature Engineering:** Create meaningful input variables.
6. **Model Selection:** Choose an algorithm (e.g., linear model, tree, SVM).
7. **Model Training:** Fit the model to the data.
8. **Evaluation:** Validate model using appropriate metrics.
9. **Deployment:** Deploy model into real-time or batch systems.
10. **Monitoring and Retraining:** Continuously check performance drift.

4. Mathematical Foundations: Linear Algebra

Linear algebra forms the core of many machine learning algorithms. Concepts include:

- **Scalars, Vectors, and Matrices:** Fundamental building blocks.
- **Matrix Multiplication:** Key in neural networks and optimization.
- **Dot Product:** Similarity measurement in NLP and vector space models.
- **Norms:** Measure of vector magnitude.
- **Eigenvalues and Eigenvectors:** Crucial for PCA, spectral clustering.

Example – Dot Product:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i$$

Matrix Multiplication (Example): Let $A \in \mathbb{R}^{2\times 3}$ and $B \in \mathbb{R}^{3\times 1}$, then $C = AB \in \mathbb{R}^{2\times 1}$.

5. Calculus Refresher: Derivatives and Gradients

- **Derivatives:** Indicate the rate of change.
- **Gradients:** Multivariate extension—used in backpropagation.
- **Chain Rule:** Vital for neural network training.

Example – Gradient Vector:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right]^T$$

Gradient Descent:

$$\theta := \theta - \alpha \cdot \nabla J(\theta)$$

Where α is the learning rate, and $J(\theta)$ is the loss function.

6. Application Areas of AI/ML

- **Healthcare:** Disease diagnosis, drug discovery
- **Finance:** Fraud detection, stock price prediction
- **Agriculture:** Crop monitoring, yield forecasting
- **Transportation:** Traffic prediction, autonomous vehicles

7. Summary of Week 1

- AI/ML/DL hierarchy clarified
- Lifecycle of machine learning projects detailed
- Mathematical foundations in linear algebra and calculus explored
- Use cases across multiple domains discussed

Week 2: Regression and Classification

1. Understanding Supervised Learning

Supervised learning refers to training algorithms on labeled datasets, where the goal is to learn a function that maps inputs to outputs. The model is trained on a dataset $(\mathbf{x}^{(i)}, y^{(i)})$, where $\mathbf{x}^{(i)}$ is the input vector and $y^{(i)}$ is the output label.

There are two main types:

- **Regression:** Predicts a continuous output variable.
- **Classification:** Predicts a discrete label.

Goal: Learn a mapping $f : \mathcal{X} \to \mathcal{Y}$ such that $\hat{y} = f(\mathbf{x})$ approximates y well on unseen data.

2. Linear Regression

Linear regression is one of the most fundamental algorithms in machine learning. It assumes a linear relationship between input features and the target output.

Model Equation:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n = \mathbf{w}^T \mathbf{x}$$

Objective: Find weights $\mathbf{w}$ that minimize the Mean Squared Error (MSE):

$$\mathcal{L}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

Gradient Descent Update Rule:

$$w_j := w_j - \alpha \cdot \frac{\partial \mathcal{L}}{\partial w_j}$$

Use Cases:

- Predicting real estate prices based on location, area, and amenities
- Estimating rainfall based on humidity and pressure
- Forecasting electricity demand from historical usage

3. Logistic Regression

Logistic regression is used when the output variable is categorical (binary or multi-class). It estimates the probability of a data point belonging to a particular class.

Sigmoid Activation Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = \mathbf{w}^T\mathbf{x}$$

The output $\hat{y} = \sigma(\mathbf{w}^T\mathbf{x})$ is a probability in the range (0, 1), which can be thresholded (e.g., 0.5) for binary classification.

4. Comparison: Linear vs Logistic Regression

Aspect	**Linear Regression**	**Logistic Regression**
Output Type	Continuous Value	Probability (0 to 1)
Function Type	Linear Function	Sigmoid Function
Loss Function	Mean Squared Error	Binary Cross-Entropy
Use Case	Predicting quantities	Classifying categories
Example	House price prediction	Email spam detection

5. Loss Functions in Depth

Loss functions quantify the difference between predicted output and the actual target value.

- **Mean Squared Error (MSE)** for regression:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Binary Cross-Entropy (BCE)** for binary classification:

$$\text{BCE} = -\left[y\log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

- **Hinge Loss** for Support Vector Machines (SVMs):

$$\text{Hinge} = \max(0, 1 - y \cdot \hat{y})$$

6. Evaluation Metrics for Classification

After training a model, it's essential to evaluate its performance using suitable metrics:

- **Accuracy:** Overall, how often is the model correct?

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Of all predicted positives, how many are truly positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** Of all actual positives, how many did the model identify?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1 Score:** Harmonic mean of precision and recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **AUC-ROC:** Measures the model's ability to distinguish between classes.

7. Visualizing Model Performance

Visual tools help assess model quality:

- **Confusion Matrix:** Tabulates true/false positives and negatives.
- **ROC Curve:** Plots True Positive Rate vs False Positive Rate.
- **Precision-Recall Curve:** Useful in imbalanced datasets.

8. Summary of Week 2

- Understood the difference between regression and classification tasks.
- Studied the theoretical and mathematical foundations of linear and logistic regression.
- Analyzed key loss functions used in model optimization.
- Learned evaluation metrics and visual techniques to judge classification performance.

Week 3: Generalization and Optimization

1. Introduction to Generalization

In machine learning, generalization is the model's ability to perform well on new, unseen data. A model that learns the training data too well may fail to generalize and thus perform poorly in practice.

Training Error vs. Test Error:

- **Training Error:** Error on the data the model was trained on.
- **Test Error:** Error on new data not seen during training.

Goal: Achieve a low test error, not just a low training error.

2. Overfitting and Underfitting

- **Underfitting:** Model is too simple to capture patterns in data (high bias).
- **Overfitting:** Model is too complex and captures noise in the data (high variance).

Example:

- Underfitting: Predicting house prices using only one feature (e.g., size).
- Overfitting: Using a high-degree polynomial for only 20 data points.

Preventing Overfitting:

- Use more training data
- Use simpler models
- Regularization
- Early stopping

3. Bias-Variance Tradeoff

Model error can be decomposed as:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Bias:** Error from incorrect model assumptions.
- **Variance:** Error from sensitivity to fluctuations in training data.
- **Irreducible Error:** Noise in the data that no model can capture.

Tradeoff:

- High bias → underfitting
- High variance → overfitting

Visual Insight:

In practice, one must balance the bias and variance by choosing an appropriate model complexity.

4. Regularization: Controlling Complexity

Regularization adds a penalty to the loss function to discourage overly complex models.

- **L1 Regularization (Lasso):**

$$\mathcal{L} = \text{Loss} + \lambda \sum_{j=1}^{n} |w_j|$$

  Encourages sparsity; sets many weights to zero.

- **L2 Regularization (Ridge):**

$$\mathcal{L} = \text{Loss} + \lambda \sum_{j=1}^{n} w_j^2$$

  Encourages small, but non-zero weights.

Why Regularize?

- Prevent overfitting
- Improve generalization
- Feature selection (L1)

5. Gradient Descent: The Core Optimizer

Gradient Descent is a first-order iterative optimization algorithm used to minimize a loss function.

Basic Update Rule:

$$\theta := \theta - \alpha \cdot \nabla J(\theta)$$

Where:

- θ: Model parameters
- α: Learning rate
- $J(\theta)$: Loss function

Algorithm Steps:

1. Initialize weights randomly.
2. Compute the gradient of the loss function.
3. Update the weights using the gradient.
4. Repeat until convergence.

6. Types of Gradient Descent

- **Batch Gradient Descent:** Uses the entire training dataset. Stable but slow on large data.
- **Stochastic Gradient Descent (SGD):** Updates weights using a single sample. Fast but noisy.
- **Mini-Batch Gradient Descent:** Compromise between batch and SGD; faster and more stable.

Mini-Batch Update Rule:

$$\theta := \theta - \alpha \cdot \frac{1}{k} \sum_{i=1}^{k} \nabla J(\theta; x^{(i)}, y^{(i)})$$

7. Adam Optimizer (Adaptive Moment Estimation)

Adam combines momentum and RMSProp. It adapts the learning rate individually for each parameter.

Key Concepts:

- Keeps track of exponentially decaying averages of past gradients (momentum).
- Also tracks squared gradients for adaptive learning.

Update Rule:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_t := \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

8. Summary of Week 3

- Generalization ensures models perform well on unseen data.
- Overfitting and underfitting reflect the tradeoff between model complexity and data.
- Bias-variance tradeoff is crucial for understanding learning capacity.
- Regularization (L1/L2) helps constrain overly complex models.
- Optimization techniques like Gradient Descent, SGD, and Adam are used to train models efficiently.

Week 4: Decision Trees and Support Vector Machines (SVM)

1. Introduction to Decision Trees

Decision Trees are supervised learning models used for both classification and regression tasks. They recursively split the dataset into subsets based on feature values to build a tree-like model of decisions.

Components:

- **Root Node:** Represents the entire dataset.
- **Internal Node:** Decision based on a feature.
- **Leaf Node:** Output or final class.
- **Branches:** Outcome of a decision.

Advantages:

- Easy to understand and interpret.
- Handles both numerical and categorical data.
- Requires little data preprocessing.

2. Entropy and Information Gain

To decide which feature to split on, decision trees use a metric like **information gain**, based on **entropy**.

Entropy: Measures impurity or uncertainty in the dataset:

$$H(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

where p_i is the proportion of class i in set S.

Information Gain:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

3. Gini Index (Alternative to Entropy)

Used in CART (Classification and Regression Trees). Defined as:

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Both entropy and Gini Index serve the same purpose: to measure impurity in a node. The one that gives better splits is used during training.

4. Tree Pruning

Overly complex trees may overfit the training data. To prevent this, pruning is used.

Types of Pruning:

- **Pre-pruning (Early Stopping):** Stop tree growth early.
- **Post-pruning:** Grow full tree and then remove unnecessary branches.

5. Random Forests

Random Forest is an ensemble learning method that builds multiple decision trees and combines their predictions.

Key Characteristics:

- Uses bagging (bootstrap aggregation).
- Reduces variance by averaging multiple tree outputs.
- Robust to overfitting compared to individual trees.

Prediction Rule:

- **Classification:** Majority voting among trees.
- **Regression:** Average of outputs.

Benefits:

- Handles high-dimensional data well.
- Resistant to overfitting.
- Works well with missing data and noisy labels.

6. Introduction to Support Vector Machines (SVM)

Support Vector Machines are powerful supervised learning models primarily used for classification, though they can also handle regression.

Goal: Find the optimal hyperplane that best separates the classes with maximum margin.

Mathematically: The decision boundary is:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = 0$$

Where $\mathbf{w}$ is the weight vector and b is the bias.

Margin:

The margin is the distance between the decision boundary and the closest data points (support vectors) from either class.

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|}$$

Maximizing margin leads to better generalization.

7. Hard vs Soft Margin

- **Hard Margin SVM:** Assumes perfect linear separation — no misclassification allowed.
- **Soft Margin SVM:** Allows some misclassification with a penalty; used in real-world noisy datasets.

Objective Function (Soft Margin):

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

Subject to:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Where:

- C is the penalty parameter for misclassification.
- ξ_i is the slack variable allowing margin violations.

8. The Kernel Trick

SVMs can also classify non-linearly separable data using the **kernel trick**. Instead of transforming data explicitly to higher dimensions, kernels compute inner products in that space efficiently.

Common Kernels:

- **Linear Kernel:** $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T\mathbf{z}$
- **Polynomial Kernel:** $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T\mathbf{z} + c)^d$
- **RBF (Gaussian) Kernel:** $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma\|\mathbf{x} - \mathbf{z}\|^2)$

Why Kernels Matter:

- Handle complex, non-linear boundaries.
- Avoid explicit high-dimensional computation.

9. Comparison: Decision Trees vs SVMs

Aspect	**Decision Trees**	**SVM**
Interpretability	High (visual structure)	Low
Handling Non-linear Data	Poor (without ensemble)	Excellent (with kernels)
Overfitting Tendency	High	Low (with regularization)
Training Time	Fast	Slower on large datasets

10. Summary of Week 4

- Decision Trees recursively split data and are highly interpretable.
- Entropy and Gini index are metrics to evaluate splits.
- Random Forests reduce variance by combining multiple trees.
- SVMs find the best separating hyperplane using margins.
- The kernel trick allows SVMs to handle complex, non-linear datasets efficiently.

Week 5: Unsupervised Learning and Clustering

1. What is Unsupervised Learning?

Unlike supervised learning, unsupervised learning deals with unlabeled data. The goal is to discover patterns, structures, or representations from data without predefined output labels.

Common Tasks:

- Clustering: Grouping similar data points
- Dimensionality Reduction: Compressing high-dimensional data
- Anomaly Detection: Identifying outliers

Applications:

- Customer segmentation in marketing
- Topic modeling in NLP
- Grouping genes with similar behavior in bioinformatics

2. Clustering Overview

Clustering aims to divide data into distinct groups such that points in the same group are more similar to each other than to points in other groups.

Types of Clustering:

- **Hard Clustering:** Each point belongs to exactly one cluster.
- **Soft Clustering:** Each point has a probability of belonging to each cluster (e.g., Gaussian Mixture Models).

3. K-Means Clustering

K-Means is a popular and efficient hard clustering algorithm.

Algorithm Steps:

1. Choose number of clusters K.
2. Initialize K cluster centroids randomly.

3. Assign each point to the nearest centroid.
4. Recompute centroids as the mean of assigned points.
5. Repeat until convergence (no change in assignments).

Objective Function: Minimize the within-cluster sum of squares (WCSS):

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Where:

- C_k is the k-th cluster
- μ_k is the centroid of cluster k

4. Choosing the Right K

Elbow Method: Plot number of clusters (K) vs WCSS. The optimal K is where the plot bends like an "elbow".

Silhouette Score: Measures how similar a point is to its own cluster compared to others. Ranges from -1 to 1.

5. Strengths and Limitations of K-Means

Strengths:

- Simple and fast for large datasets
- Works well when clusters are well-separated

Limitations:

- Sensitive to initial centroid positions
- Assumes spherical clusters
- Fails on complex shapes or varying densities

6. Principal Component Analysis (PCA)

PCA is a powerful unsupervised learning technique used for dimensionality reduction. It projects high-dimensional data into a lower-dimensional space while retaining most of the variance (information).

Why Dimensionality Reduction?

- Reduces computational complexity
- Removes noise and redundant features
- Helps in visualization (e.g., 2D plot of 100D data)

Mathematical Intuition:

PCA finds new axes (principal components) such that:

- The first principal component captures the direction of maximum variance.
- Subsequent components are orthogonal and capture the remaining variance.

Steps to Perform PCA:

1. Standardize the data (mean = 0, variance = 1).
2. Compute the covariance matrix:

$$\text{Cov}(X) = \frac{1}{n-1} X^T X$$

3. Compute eigenvectors and eigenvalues of the covariance matrix.
4. Select the top k eigenvectors (principal components).
5. Project data onto the new subspace:

$$Z = XW$$

Where W is the matrix of selected eigenvectors.

7. Eigenvalues and Eigenvectors (Review)

- An **eigenvector** of a square matrix A is a non-zero vector $\mathbf{v}$ such that:

$$A\mathbf{v} = \lambda \mathbf{v}$$

- λ is called the **eigenvalue** corresponding to eigenvector $\mathbf{v}$.

In PCA, eigenvectors represent directions (components), and eigenvalues represent the amount of variance captured in each direction.

8. Visualizing PCA

- In 2D: PCA rotates the axes so the new x-axis captures the most variance.
- In higher dimensions: PCA uncovers directions along which data is spread the most.

Note: PCA is a linear technique — it cannot capture complex nonlinear structures.

9. PCA Applications

- Preprocessing step before clustering or classification
- Noise reduction
- Facial recognition (eigenfaces)
- Genomics and neuroscience for high-dimensional biological data

10. Summary of Week 5

- Unsupervised learning helps identify hidden patterns without labels.
- K-Means clustering groups similar points based on distance to centroids.
- Evaluation techniques like the Elbow method and silhouette score assist in choosing optimal clusters.
- PCA reduces dimensionality while preserving maximum variance using eigenvectors and eigenvalues.
- Together, clustering and PCA form the backbone of many exploratory data analysis pipelines.

Week 6: Neural Networks and Deep Learning Basics

1. Biological and Artificial Neurons

Neural Networks are inspired by the structure of the human brain. They are composed of interconnected units called artificial neurons.

Biological Analogy:

- Neurons receive input through dendrites.
- They process and transmit signals through synapses.
- Firing depends on whether a threshold is exceeded.

Artificial Neuron (Perceptron): An artificial neuron computes a weighted sum of inputs and applies an activation function:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

$$\hat{y} = f(z)$$

2. Perceptron Algorithm

The perceptron is a binary classifier. It updates weights based on prediction error.

Update Rule:

$$w_j := w_j + \alpha(y - \hat{y})x_j$$

Limitations:

- Can only solve linearly separable problems.
- Cannot learn XOR function.

3. Multi-Layer Perceptron (MLP)

To solve non-linear problems, we use Multi-Layer Perceptrons (MLPs) with:

- Input layer
- Hidden layer(s)
- Output layer

MLP Function:

$$\hat{y} = f(W^{(2)} f(W^{(1)}\mathbf{x} + b^{(1)}) + b^{(2)})$$

Each layer transforms the data through learned weights and non-linear activation functions.

4. Activation Functions

Activation functions introduce non-linearity, enabling the network to model complex relationships.

- **Sigmoid:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

  Smooth, outputs in (0,1), but suffers from vanishing gradients.

- **Tanh:**

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

  Output in (-1, 1), zero-centered.

- **ReLU (Rectified Linear Unit):**

$$f(z) = \max(0, z)$$

  Sparse activation, fast computation, widely used in deep learning.

- **Leaky ReLU:**

$$f(z) = \max(\alpha z, z), \quad \text{where } \alpha \ll 1$$

  Solves the dying ReLU problem by allowing a small gradient when $z < 0$.

5. Forward Propagation

Forward propagation is the process of passing input data through the network to obtain predictions.

Steps in a Single Hidden Layer MLP:

1. Compute weighted sum at hidden layer:

$$z^{(1)} = W^{(1)}\mathbf{x} + b^{(1)}$$

2. Apply activation:

$$a^{(1)} = f(z^{(1)})$$

3. Compute output layer input:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

4. Apply activation to get prediction:

$$\hat{y} = f(z^{(2)})$$

6. Backpropagation: Training Neural Networks

Backpropagation computes the gradient of the loss function with respect to each weight using the chain rule of calculus.

Steps:

1. Compute output error:

$$\delta^{(2)} = \hat{y} - y$$

2. Compute gradients for output layer:

$$\nabla W^{(2)} = \delta^{(2)}(a^{(1)})^T$$

3. Propagate error to hidden layer:

$$\delta^{(1)} = (W^{(2)})^T\delta^{(2)} \cdot f'(z^{(1)})$$

4. Compute gradients for hidden layer:

$$\nabla W^{(1)} = \delta^{(1)}\mathbf{x}^T$$

Weights are updated using gradient descent:

$$W := W - \alpha\nabla W$$

Note: The choice of loss function depends on the task (e.g., cross-entropy for classification, MSE for regression).

7. Neural Network Training Concepts

- **Epoch:** One full pass through the training dataset.
- **Batch Size:** Number of samples per gradient update.
- **Learning Rate (α):** Controls the size of weight updates.
- **Loss Function:** Measures prediction error.
- **Optimizer:** Algorithm to minimize loss (e.g., SGD, Adam).

8. Challenges in Training Neural Networks

- **Vanishing/Exploding Gradients:** Especially in deep networks.
- **Overfitting:** Handled using dropout, regularization, and early stopping.
- **Hyperparameter Tuning:** Choosing learning rate, batch size, and number of layers/neurons.

9. Summary of Week 6

- Neural networks consist of layers of artificial neurons.
- Perceptrons can be stacked to model non-linear functions (MLPs).
- Activation functions bring non-linearity into the model.
- Forward propagation computes predictions.
- Backpropagation calculates gradients and updates weights.
- Proper training requires balancing architecture, learning rate, and regularization.

Week 7: Convolutional Neural Networks and Computer Vision

1. Introduction to CNNs

Convolutional Neural Networks (CNNs) are specialized neural networks designed for processing data with grid-like topology, such as images.

Why CNNs for Images?

- Standard MLPs require flattening images into 1D vectors, which loses spatial structure.
- CNNs preserve local spatial relationships using convolution operations.
- They reduce the number of parameters significantly using weight sharing.

Applications:

- Image classification
- Object detection
- Face recognition
- Medical image analysis

2. CNN Architecture Overview

A typical CNN is made of the following layers:

- **Convolutional Layers:** Detect features using filters.
- **Activation Function (e.g., ReLU):** Adds non-linearity.
- **Pooling Layers:** Reduce spatial dimensions.
- **Fully Connected (FC) Layers:** Perform final classification.

3. Convolution Operation

Convolution applies a kernel (or filter) to input data to extract features such as edges, corners, textures, etc.

Mathematical Definition: Given input I and kernel K:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

Key Parameters:

- **Kernel Size:** Size of the filter (e.g., 3×3)
- **Stride:** Number of pixels by which the filter moves
- **Padding:** Adding borders to preserve size

Feature Maps:

Each convolution creates a 2D feature map representing presence of specific patterns across the image.

Note: Multiple filters are used to detect different features in parallel.

4. Activation: ReLU

ReLU (Rectified Linear Unit) is commonly applied after each convolution:

$$f(x) = \max(0, x)$$

It speeds up convergence and introduces non-linearity.

5. Pooling Layers

Pooling is a downsampling technique that reduces the spatial dimensions (width and height) of feature maps.

Why Pooling?

- Reduces computation
- Controls overfitting
- Introduces translation invariance

Types of Pooling:

- **Max Pooling:** Takes the maximum value in a patch
- **Average Pooling:** Takes the average of values in a patch

Example (Max Pooling with 2×2 window):

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \rightarrow \max(1, 2, 3, 4) = 4$$

6. Flattening and Fully Connected Layers

After several convolution and pooling operations, the feature maps are flattened into a 1D vector.

- **Flattening:** Converts 2D feature maps into a single vector
- **Fully Connected Layers (FC):** Perform the final classification based on high-level features

Output: The final FC layer typically uses a Softmax activation for multi-class classification:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

7. Dropout Regularization

Dropout is a technique to reduce overfitting by randomly disabling a fraction of neurons during training.

- During each iteration, some nodes are dropped with a probability p.
- At test time, all nodes are used with scaled weights.

Dropout Example: If $p = 0.5$, each neuron has a 50% chance of being ignored during one training pass.

8. Training CNNs

The training process for CNNs is similar to other neural networks:

1. Initialize weights
2. Forward propagate inputs through layers
3. Compute loss (e.g., cross-entropy)
4. Backpropagate errors and compute gradients
5. Update weights using optimizers like Adam

9. Common CNN Architectures

- **LeNet-5 (1998):** First CNN used for digit recognition.
- **AlexNet (2012):** Won ImageNet with deep architecture + ReLU + GPU.
- **VGGNet:** Used only 3×3 convolutions, very deep.
- **ResNet:** Introduced skip connections for very deep networks.

10. Summary of Week 7

- CNNs are ideal for image data due to spatial feature extraction.
- Convolutional layers use filters to detect local patterns.
- Pooling reduces size and variance.
- Fully connected layers handle final classification.
- Dropout helps reduce overfitting.
- CNNs are the core of modern computer vision systems.

Week 8: Project Work and Comprehensive Review

1. Final Project Integration

The objective of Week 8 is to consolidate all learned concepts into a unified AI/ML pipeline and apply them toward a real or hypothetical project.

2. Designing an ML System: Step-by-Step

Problem Statement: Define a clear objective. **Example:** "Predict whether a bank customer will churn based on transaction history."

1. **Data Collection:**
   - Source from APIs, databases, or sensors.
   - Consider ethics and privacy.
2. **Data Preprocessing:**
   - Handle missing values, outliers.
   - Normalize or standardize features.
3. **Exploratory Data Analysis (EDA):**
   - Use statistics and visualizations to understand distributions.
4. **Feature Engineering:**
   - Extract useful features.
   - Reduce dimensionality with PCA (Week 5).
5. **Model Selection:**
   - For structured data: Decision Trees, Random Forests, SVM (Week 4)
   - For vision tasks: CNN (Week 7)
   - For general tabular data: Logistic Regression, Neural Network (Week 2,6)
6. **Training:**
   - Use Gradient Descent/Adam (Week 3,6)

- Monitor training/validation loss

7. **Evaluation:**
   - Use metrics like Accuracy, F1, AUC (Week 2)
8. **Model Optimization:**
   - Tune hyperparameters, apply regularization, reduce overfitting (Week 3,6)
9. **Deployment:**
   - Export model using libraries (e.g., ONNX, TensorFlow Lite)
   - Integrate with web/mobile apps

3. Common Tools and Frameworks

- **Python Libraries:** scikit-learn, pandas, numpy
- **Deep Learning Frameworks:** TensorFlow, PyTorch, Keras
- **Visualization:** matplotlib, seaborn
- **Deployment:** Flask, FastAPI, Docker

4. Weekly Topic Summary Table

Week	Key Concepts
Week 1	AI vs ML vs DL, Types of Learning, Linear Algebra (Matrices, Vectors), Derivatives, Gradient Descent
Week 2	Linear and Logistic Regression, Loss Functions (MSE, Cross-Entropy), Classification Metrics (Accuracy, Precision, Recall, F1)
Week 3	Overfitting, Underfitting, Bias-Variance Tradeoff, L1/L2 Regularization, Optimization: Gradient Descent, SGD, Adam
Week 4	Decision Trees (Entropy, Gini), Random Forests, Support Vector Machines (SVM), Kernel Trick
Week 5	Clustering (K-Means), Elbow Method, Dimensionality Reduction (PCA), Eigenvectors, Eigenvalues
Week 6	Perceptron, Multi-Layer Neural Networks, Activation Functions (ReLU, Sigmoid, Tanh), Forward/Backward Propagation
Week 7	Convolutional Neural Networks (CNNs), Convolution, Pooling, Dropout, Fully Connected Layers, Applications in Vision
Week 8	Full ML Pipeline (EDA $\rightarrow$ Deployment), Project Integration, Toolkits (scikit-learn, PyTorch), Final Reflections

5. Final Reflections

- Machine learning is a multidisciplinary field that blends mathematics, computer science, and domain knowledge.
- Hands-on understanding of concepts like loss minimization, regularization, and back-propagation is essential to develop robust ML systems.
- Each component in the pipeline — from data collection to model deployment — plays a critical role in the system's performance and generalizability.
- This summer project has not only introduced theoretical foundations but also helped me envision real-world applications and career pathways in AI/ML.

6. Learning Outcomes

After completing this 8-week learning journey, I have gained:

- A strong foundational understanding of AI, ML, and DL theory.
- Familiarity with essential algorithms in supervised and unsupervised learning.
- Insight into deep learning architectures including CNNs.
- Awareness of practical considerations in training, tuning, and deploying models.

7. Next Steps

- Apply these concepts in hands-on projects and Kaggle competitions.
- Learn more advanced topics like RNNs, Transformers, GANs.
- Explore real-world problems using open datasets.
- Continue research in theoretical ML topics such as VC dimension, convergence proofs, and optimization theory.

References:

1. *Artificial Intelligence: A Modern Approach* [Stuart Russell, Peter Norvig]
2. *Pattern Recognition and Machine Learning* [Christopher M. Bishop]
3. *Deep Learning* [Ian Goodfellow, Yoshua Bengio, Aaron Courville]
4. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* [Aurélien Géron]
5. *The Hundred-Page Machine Learning Book* [Andriy Burkov]
6. *Neural Networks and Deep Learning* [Michael Nielsen] (Online)
7. ”Artificial Intelligence and Machine Learning” [YouTube Video]